

Fast Sector Decomposition from Black-Box Symanzik Polynomials

1 The example triangle

We consider the scalar one-loop triangle with one incoming off-shell momentum and two outgoing massless momenta,

$$p_0 = p_1 + p_2, \quad p_1^2 = p_2^2 = 0, \quad p_0^2 = s = m_0^2. \quad (1)$$

All three internal propagators are taken to have the same mass m . A convenient routing is

$$D_0 = \ell^2 - m^2 + i0, \quad (2)$$

$$D_1 = (\ell + p_1)^2 - m^2 + i0, \quad (3)$$

$$D_2 = (\ell - p_2)^2 - m^2 + i0. \quad (4)$$

The scalar integral is

$$C_0(s; m^2) = \int \frac{d^D \ell}{i\pi^{D/2}} \frac{1}{D_0 D_1 D_2}. \quad (5)$$

With Feynman parameters $x_i \geq 0$, the projective representation is

$$C_0(s; m^2) = -\Gamma(\alpha) \int_0^\infty dx_0 dx_1 dx_2 \delta(1 - x_0 - x_1 - x_2) \frac{\mathcal{U}^{3-D}}{\mathcal{F}^\alpha}, \quad \alpha = 3 - \frac{D}{2}. \quad (6)$$

For this routing,

$$\mathcal{U} = x_0 + x_1 + x_2, \quad (7)$$

and

$$\mathcal{F} = m^2 \mathcal{U}^2 - s x_1 x_2 - i0. \quad (8)$$

Solving the simplex constraint by

$$x_0 = 1 - x_1 - x_2 \quad (9)$$

gives

$$0 \leq x_1 \leq 1, \quad 0 \leq x_2 \leq 1 - x_1, \quad (10)$$

and therefore

$$\mathcal{U} = 1, \quad \mathcal{F} = m^2 - s x_1 x_2 - i0. \quad (11)$$

For $D = 4 - 2\epsilon$, one has $\alpha = 1 + \epsilon$, and

$$C_0(s; m^2) = -\Gamma(1 + \epsilon) \int_0^1 dx_1 \int_0^{1-x_1} dx_2 (m^2 - s x_1 x_2 - i0)^{-1-\epsilon}. \quad (12)$$

Throughout the rest of the note, sector integrands I_s do not include the global prefactor $-\Gamma(\alpha)$. Thus an exact decomposition has the form

$$C_0(s; m^2) = -\Gamma(\alpha) \sum_s \int_{\mathcal{C}_s} dy_s I_s(y_s). \quad (13)$$

1.1 Endpoint sectors

The primary endpoint decomposition splits the simplex into two sectors.

Sector S_1 , $x_1 \geq x_2$.

$$X_{S_1}(t, z) = (x_0, x_1, x_2) = \left(1 - t, \frac{t}{1+z}, \frac{tz}{1+z}\right), \quad 0 \leq t, z \leq 1. \quad (14)$$

The Jacobian is

$$J_{S_1}(t, z) = \frac{t}{(1+z)^2}. \quad (15)$$

The pulled-back second Symanzik polynomial is

$$\mathcal{F}_{S_1}(t, z) = m^2 - s \frac{t^2 z}{(1+z)^2} - i0. \quad (16)$$

In the massless endpoint case $m = 0$,

$$\mathcal{F}_{S_1}(t, z) = (-s - i0) \frac{t^2 z}{(1+z)^2}. \quad (17)$$

The expected monomial is therefore

$$M_{S_1}(t, z) = t^2 z. \quad (18)$$

Sector S_2 , $x_2 \geq x_1$.

$$X_{S_2}(t, z) = (x_0, x_1, x_2) = \left(1 - t, \frac{tz}{1+z}, \frac{t}{1+z}\right), \quad 0 \leq t, z \leq 1. \quad (19)$$

The Jacobian is again

$$J_{S_2}(t, z) = \frac{t}{(1+z)^2}. \quad (20)$$

The pulled-back polynomial is

$$\mathcal{F}_{S_2}(t, z) = m^2 - s \frac{t^2 z}{(1+z)^2} - i0. \quad (21)$$

In the massless endpoint case,

$$\mathcal{F}_{S_2}(t, z) = (-s - i0) \frac{t^2 z}{(1+z)^2}, \quad M_{S_2}(t, z) = t^2 z. \quad (22)$$

With these definitions, the two-sector representation of the original triangle is

$$C_0(s; m^2) = -\Gamma(\alpha) \sum_{a=1}^2 \int_0^1 dt \int_0^1 dz I_{S_a}(t, z), \quad (23)$$

$$I_{S_a}(t, z) = \frac{t}{(1+z)^2} \mathcal{U}(X_{S_a}(t, z))^{3-D} \mathcal{F}(X_{S_a}(t, z))^{-\alpha}. \quad (24)$$

Equivalently, after inserting the explicit polynomial,

$$C_0(s; m^2) = -\Gamma(\alpha) \sum_{a=1}^2 \int_0^1 dt \int_0^1 dz \frac{t}{(1+z)^2} \left(m^2 - s \frac{t^2 z}{(1+z)^2} - i0 \right)^{-\alpha}. \quad (25)$$

For $D = 4 - 2\epsilon$ and $m = 0$, this becomes

$$C_0(s; 0) = -\Gamma(1 + \epsilon) \sum_{a=1}^2 \int_0^1 dt \int_0^1 dz t^{-1-2\epsilon} z^{-1-\epsilon} G_{S_a}(t, z), \quad (26)$$

where

$$G_{S_a}(t, z) = (-s - i0)^{-1-\epsilon} (1+z)^{2\epsilon}. \quad (27)$$

2 Direct approach

In the direct approach, each sector map X_s is substituted explicitly into $\mathcal{U}$ and $\mathcal{F}$. If the pulled-back polynomial factorizes as

$$\mathcal{F}_s(y) = \mathcal{F}(X_s(y)) = M_s(y) \phi_s(y), \quad (28)$$

then the sector integrand is built as

$$I_s^{\text{dir}}(y) = J_s(y) \mathcal{U}(X_s(y))^{3-D} M_s(y)^{-\alpha} \phi_s(y)^{-\alpha}. \quad (29)$$

The corresponding contribution to the original scalar integral is always understood as

$$C_0 = -\Gamma(\alpha) \sum_s \int_{\mathcal{C}_s} dy_s I_s^{\text{dir}}(y_s), \quad (30)$$

with the sector domains $\mathcal{C}_s$ specified explicitly below.

2.1 Endpoint sector S_1

For $m = 0$,

$$\mathcal{U}_{S_1}(t, z) = 1, \quad (31)$$

and

$$\mathcal{F}_{S_1}(t, z) = (-s - i0) \frac{t^2 z}{(1+z)^2} = M_{S_1}(t, z) \phi_{S_1}(t, z), \quad (32)$$

with

$$M_{S_1}(t, z) = t^2 z, \quad \phi_{S_1}(t, z) = \frac{-s - i0}{(1+z)^2}. \quad (33)$$

Therefore

$$I_{S_1}^{\text{dir}}(t, z) = \frac{t}{(1+z)^2} (t^2 z)^{-\alpha} \left(\frac{-s - i0}{(1+z)^2} \right)^{-\alpha}. \quad (34)$$

For $D = 4 - 2\epsilon$, i.e. $\alpha = 1 + \epsilon$,

$$I_{S_1}^{\text{dir}}(t, z) = (-s - i0)^{-1-\epsilon} t^{-1-2\epsilon} z^{-1-\epsilon} (1+z)^{2\epsilon}. \quad (35)$$

2.2 Endpoint sector S_2

By symmetry,

$$\mathcal{F}_{S_2}(t, z) = (-s - i0) \frac{t^2 z}{(1+z)^2}, \quad M_{S_2}(t, z) = t^2 z, \quad (36)$$

and hence

$$I_{S_2}^{\text{dir}}(t, z) = (-s - i0)^{-1-\epsilon} t^{-1-2\epsilon} z^{-1-\epsilon} (1+z)^{2\epsilon}. \quad (37)$$

The full massless triangle is therefore

$$\begin{aligned} C_0(s; 0) &= -\Gamma(1+\epsilon) \sum_{a=1}^2 \int_0^1 dt \int_0^1 dz I_{S_a}^{\text{dir}}(t, z) \\ &= -2\Gamma(1+\epsilon) (-s - i0)^{-1-\epsilon} \int_0^1 dt \int_0^1 dz t^{-1-2\epsilon} z^{-1-\epsilon} (1+z)^{2\epsilon}. \end{aligned} \quad (38)$$

3 Indirect approach

The indirect approach keeps the Symanzik polynomials closed. We assume only black-box evaluators

$$\widehat{\mathcal{U}}(x_0, x_1, x_2), \quad \widehat{\mathcal{F}}(x_0, x_1, x_2), \quad (39)$$

together with their partial derivatives with respect to the original parameters x_i . In practice, these derivative jets can be computed efficiently with dual numbers.

For a sector map X_s , define the pullbacks

$$U_s(y) = \widehat{\mathcal{U}}(X_s(y)), \quad F_s(y) = \widehat{\mathcal{F}}(X_s(y)). \quad (40)$$

Derivatives with respect to sector variables are obtained by chain rules. For example,

$$\partial_a F_s(y) = \sum_i \frac{\partial \widehat{\mathcal{F}}}{\partial x_i}(X_s(y)) \frac{\partial X_{s,i}}{\partial y_a}(y), \quad (41)$$

and

$$\begin{aligned} \partial_a \partial_b F_s(y) &= \sum_{i,j} \frac{\partial^2 \widehat{\mathcal{F}}}{\partial x_i \partial x_j}(X_s(y)) \frac{\partial X_{s,i}}{\partial y_a}(y) \frac{\partial X_{s,j}}{\partial y_b}(y) \\ &\quad + \sum_i \frac{\partial \widehat{\mathcal{F}}}{\partial x_i}(X_s(y)) \frac{\partial^2 X_{s,i}}{\partial y_a \partial y_b}(y). \end{aligned} \quad (42)$$

Higher derivatives are obtained similarly, or more directly by propagating dual-number jets through the known sector map and then through the black-box polynomial evaluator.

The object to construct is the regular coefficient ϕ_s defined by

$$F_s(y) = M_s(y) \phi_s(y), \quad (43)$$

where M_s is the expected monomial for that sector. Once ϕ_s is available, the sector integrand is

$$I_s^{\text{ind}}(y) = J_s(y) U_s(y)^{3-D} M_s(y)^{-\alpha} \phi_s(y)^{-\alpha}. \quad (44)$$

At interior points this is pointwise identical to direct substitution, since

$$\phi_s(y) = \frac{\widehat{\mathcal{F}}(X_s(y))}{M_s(y)} \quad (45)$$

there. For the endpoint decomposition before any monomial extraction, the black-box form of the exact triangle is

$$C_0(s; m^2) = -\Gamma(\alpha) \sum_{a=1}^2 \int_0^1 dt \int_0^1 dz \frac{t}{(1+z)^2} \widehat{\mathcal{U}}(X_{S_a}(t, z))^{3-D} \widehat{\mathcal{F}}(X_{S_a}(t, z))^{-\alpha}. \quad (46)$$

When $m = 0$, and hence $\widehat{\mathcal{F}}(X_{S_a}(t, z)) = t^2 z \phi_{S_a}(t, z)$, this same equality becomes

$$\begin{aligned} C_0(s; 0) &= -\Gamma(1+\epsilon) \sum_{a=1}^2 \int_0^1 dt \int_0^1 dz \frac{t}{(1+z)^2} \widehat{\mathcal{U}}(X_{S_a}(t, z))^{-1+2\epsilon} \\ &\quad \times (t^2 z)^{-1-\epsilon} \phi_{S_a}(t, z)^{-1-\epsilon}. \end{aligned} \quad (47)$$

3.1 Using subtraction terms

The subtraction construction is the one used to expose the Laurent poles in ϵ . At interior points one evaluates

$$\phi_s(y) = \frac{\widehat{\mathcal{F}}(X_s(y))}{M_s(y)}. \quad (48)$$

Boundary values are supplied by the continuous extension, using derivatives. For the first endpoint sector,

$$\phi_{S_1}(0, z) = \frac{1}{2z} [\partial_T^2 F_{S_1}(T, z)]_{T=0}, \quad (49)$$

$$\phi_{S_1}(t, 0) = \frac{1}{t^2} [\partial_Z F_{S_1}(t, Z)]_{Z=0}, \quad (50)$$

$$\phi_{S_1}(0, 0) = \frac{1}{2} [\partial_T^2 \partial_Z F_{S_1}(T, Z)]_{T=0, Z=0}. \quad (51)$$

Again, the derivatives of F_{S_1} are not symbolic derivatives of an opened polynomial. They are derivatives of $\widehat{\mathcal{F}} \circ X_{S_1}$, obtained from the black-box derivative jets and the chain rule.

For $D = 4 - 2\epsilon$, write each factorized endpoint integrand as

$$I_{S_a}^{\text{ind}}(t, z) = t^{-1-2\epsilon} z^{-1-\epsilon} G_{S_a}(t, z), \quad a = 1, 2, \quad (52)$$

where

$$G_{S_a}(t, z) = \frac{1}{(1+z)^2} U_{S_a}(t, z)^{-1+2\epsilon} \phi_{S_a}(t, z)^{-1-\epsilon}. \quad (53)$$

The two-dimensional endpoint subtraction identity is

$$\begin{aligned} & \int_0^1 dt \int_0^1 dz t^{-1-2\epsilon} z^{-1-\epsilon} G(t, z) \\ &= \int_0^1 dt \int_0^1 dz t^{-1-2\epsilon} z^{-1-\epsilon} [G(t, z) - G(0, z) - G(t, 0) + G(0, 0)] \\ & \quad - \frac{1}{2\epsilon} \int_0^1 dz z^{-1-\epsilon} [G(0, z) - G(0, 0)] \\ & \quad - \frac{1}{\epsilon} \int_0^1 dt t^{-1-2\epsilon} [G(t, 0) - G(0, 0)] + \frac{G(0, 0)}{2\epsilon^2}. \end{aligned} \quad (54)$$

Each bracket is less singular at the corresponding endpoint. This is the mechanism that turns endpoint singularities into explicit poles and finite integrals. Applied sector by sector, it gives the full massless triangle as

$$\begin{aligned} C_0(s; 0) &= -\Gamma(1+\epsilon) \sum_{a=1}^2 \left\{ \int_0^1 dt \int_0^1 dz t^{-1-2\epsilon} z^{-1-\epsilon} \left[G_{S_a}(t, z) - G_{S_a}(0, z) \right. \right. \\ & \quad \left. \left. - G_{S_a}(t, 0) + G_{S_a}(0, 0) \right] \right. \\ & \quad - \frac{1}{2\epsilon} \int_0^1 dz z^{-1-\epsilon} [G_{S_a}(0, z) - G_{S_a}(0, 0)] \\ & \quad \left. - \frac{1}{\epsilon} \int_0^1 dt t^{-1-2\epsilon} [G_{S_a}(t, 0) - G_{S_a}(0, 0)] + \frac{G_{S_a}(0, 0)}{2\epsilon^2} \right\}. \end{aligned} \quad (55)$$

For the explicit massless triangle in sector S_1 ,

$$U_{S_1}(t, z) = 1, \quad \phi_{S_1}(t, z) = \frac{-s - i0}{(1+z)^2}. \quad (56)$$

Thus

$$G_{S_1}^{\text{exp}}(t, z) = (-s - i0)^{-1-\epsilon}(1+z)^{2\epsilon}. \quad (57)$$

This function is independent of t . Therefore

$$G_{S_1}^{\text{exp}}(t, z) - G_{S_1}^{\text{exp}}(0, z) - G_{S_1}^{\text{exp}}(t, 0) + G_{S_1}^{\text{exp}}(0, 0) = 0, \quad (58)$$

and

$$G_{S_1}^{\text{exp}}(t, 0) - G_{S_1}^{\text{exp}}(0, 0) = 0. \quad (59)$$

The only non-trivial subtraction piece is

$$G_{S_1}^{\text{exp}}(0, z) - G_{S_1}^{\text{exp}}(0, 0) = (-s - i0)^{-1-\epsilon} [(1+z)^{2\epsilon} - 1]. \quad (60)$$

The explicitly subtracted result for one endpoint sector is therefore

$$\begin{aligned} & \int_0^1 dt \int_0^1 dz I_{S_1}^{\text{dir}}(t, z) \\ &= -\frac{(-s - i0)^{-1-\epsilon}}{2\epsilon} \int_0^1 dz z^{-1-\epsilon} [(1+z)^{2\epsilon} - 1] + \frac{(-s - i0)^{-1-\epsilon}}{2\epsilon^2}. \end{aligned} \quad (61)$$

The second sector gives the same contribution, so the full massless triangle is

$$C_0(s; 0) = -2\Gamma(1+\epsilon) \left\{ -\frac{(-s - i0)^{-1-\epsilon}}{2\epsilon} \int_0^1 dz z^{-1-\epsilon} [(1+z)^{2\epsilon} - 1] + \frac{(-s - i0)^{-1-\epsilon}}{2\epsilon^2} \right\}. \quad (62)$$

This expression was obtained without requiring any symbolic expansion in an implementation: only X_{S_a} , J_{S_a} , the known monomial $t^2 z$, and black-box values and derivative limits of $\hat{\mathcal{U}}$ and $\hat{\mathcal{F}}$ are required in each endpoint sector.

4 General parametric structure

For a scalar L -loop integral with N propagators of powers ν_i , define

$$A = \sum_{i=1}^N \nu_i, \quad D = D_0 + D\epsilon. \quad (63)$$

With the usual projective Feynman-parameter normalization, the parametric integral has the schematic form

$$\begin{aligned} I(\nu) = \mathcal{P}(D, \nu) \frac{\Gamma(A - LD/2)}{\prod_i \Gamma(\nu_i)} \int_{x_i \geq 0} \left[\prod_{i=1}^N dx_i x_i^{\nu_i-1} \right] \delta\left(1 - \sum_i x_i\right) \\ \times \mathcal{U}(x)^{A-(L+1)D/2} \mathcal{F}(x)^{-(A-LD/2)}. \end{aligned} \quad (64)$$

The convention-dependent prefactor $\mathcal{P}(D, \nu)$ contains the signs, powers of $i\pi^{D/2}$, and any global normalization choices. It is not part of the sector-local black-box construction. What the sector processor needs are the affine powers

$$p_U(\epsilon) = A - (L+1)D/2, \quad (65)$$

$$p_F(\epsilon) = -(A - LD/2), \quad (66)$$

the parameter weights $\nu_i - 1$, and the retained black-box evaluators for $\mathcal{U}$ and $\mathcal{F}$.

After a sector map $x = X_s(y)$, the topology-specific sector generator should separate each possible endpoint monomial source,

$$J_s(y) \prod_i X_{s,i}(y)^{\nu_i-1} N_s(y) = J_{s,\text{reg}}(y) \prod_a y_a^{j_{s,a}}, \quad (67)$$

$$\mathcal{U}(X_s(y)) = M_{U,s}(y) \psi_s(y), \quad M_{U,s}(y) = \prod_a y_a^{u_{s,a}}, \quad (68)$$

$$\mathcal{F}(X_s(y)) = M_{F,s}(y) \phi_s(y), \quad M_{F,s}(y) = \prod_a y_a^{f_{s,a}}. \quad (69)$$

Here N_s denotes any smooth numerator or extra parameter-space factor. The regular functions ψ_s and ϕ_s are evaluated by black-box dual-number limits of $\mathcal{U}$ and $\mathcal{F}$; neither polynomial is symbolically opened inside the processor. The full endpoint power of y_a is then

$$\rho_{s,a}(\epsilon) = j_{s,a} + u_{s,a} p_U(\epsilon) + f_{s,a} p_F(\epsilon), \quad (70)$$

with any additional numerator or measure monomial powers included in $j_{s,a}$. This is the object that decides the subtraction problem. The implemented examples are logarithmic endpoint cases,

$$\rho_{s,a}(\epsilon) = -1 + c_{s,a}\epsilon, \quad c_{s,a} \neq 0, \quad (71)$$

which produce explicit Laurent poles from $\int_0^1 dy_a y_a^{-1+c_{s,a}\epsilon}$.

For a uniform notation, let

$$P \in \{\mathcal{U}, \mathcal{F}\}, \quad r_{P,s,a} \in \mathbb{N}, \quad M_{P,s}(y) = \prod_{a \in A_s} y_a^{r_{P,s,a}}, \quad (72)$$

with $r_{\mathcal{U},s,a} = u_{s,a}$, $r_{\mathcal{F},s,a} = f_{s,a}$, and A_s the set of sector axes on which an endpoint monomial has been declared. Define

$$P_s(y) = \hat{P}(X_s(y)), \quad R_{P,s}(y) = \frac{P_s(y)}{M_{P,s}(y)}. \quad (73)$$

Thus $R_{\mathcal{U},s} = \psi_s$ and $R_{\mathcal{F},s} = \phi_s$. At an interior point this is just the quotient above. At a boundary point, let

$$B(y) = \{a \in A_s \mid y_a = 0\}, \quad \bar{B} = A_s \setminus B, \quad (74)$$

and introduce dual variables η_b for $b \in B$. The boundary jet is formed by

$$\hat{y}_a(\eta) = \begin{cases} \eta_a, & a \in B, \\ y_a, & a \notin B. \end{cases} \quad (75)$$

Then the regular residual is the Taylor coefficient

$$R_{P,s}(y) = \frac{[\prod_{b \in B} \eta_b^{r_{P,s,b}}] \hat{P}(X_s(\hat{y}(\eta)))}{\prod_{a \in \bar{B}} y_a^{r_{P,s,a}}}. \quad (76)$$

Here $[\eta^n]$ denotes the Taylor coefficient, not the derivative. In derivative notation the same statement is

$$R_{P,s}(y) = \frac{\left(\prod_{b \in B} \frac{1}{r_{P,s,b}!} \partial_{\eta_b}^{r_{P,s,b}} \right) \hat{P}(X_s(\hat{y}(\eta))) \Big|_{\eta=0}}{\prod_{a \in \bar{B}} y_a^{r_{P,s,a}}}. \quad (77)$$

The chain rule is applied at the level of truncated jets:

$$J_B^{\mathbf{r}}(P_s)(y) = J_x^{\mathbf{r}}(\hat{P})(X_s(y)) \circ J_B^{\mathbf{r}}(X_s)(y), \quad (78)$$

where

$$\mathbf{r} = (r_{P,s,b})_{b \in B}. \quad (79)$$

Equivalently, for the first two derivative orders,

$$\partial_a P_s(y) = \sum_i \frac{\partial \hat{P}}{\partial x_i}(X_s(y)) \partial_a X_{s,i}(y), \quad (80)$$

$$\begin{aligned} \partial_a \partial_b P_s(y) &= \sum_{i,j} \frac{\partial^2 \hat{P}}{\partial x_i \partial x_j}(X_s(y)) \partial_a X_{s,i}(y) \partial_b X_{s,j}(y) \\ &+ \sum_i \frac{\partial \hat{P}}{\partial x_i}(X_s(y)) \partial_a \partial_b X_{s,i}(y). \end{aligned} \quad (81)$$

After extracting the declared monomials, every sector has the form

$$I_s(\epsilon) = \int_{[0,1]^d} d^d y \left[\prod_{a \in A_s} y_a^{\rho_{s,a}(\epsilon)} \right] G_s(y, \epsilon), \quad (82)$$

where

$$G_s(y, \epsilon) = J_{s,\text{reg}}(y) R_{\mathcal{U},s}(y)^{p_U(\epsilon)} R_{\mathcal{F},s}(y)^{p_F(\epsilon)} \quad (83)$$

times any remaining regular numerator factor. Write

$$\rho_{s,a}(\epsilon) = \beta_{s,a} + c_{s,a}\epsilon, \quad \beta_{s,a} \in \mathbb{Z}, \quad (84)$$

and define the endpoint Taylor order

$$N_{s,a} = \max(-\beta_{s,a} - 1, 0). \quad (85)$$

For one endpoint variable,

$$\begin{aligned} \int_0^1 dy y^{\beta+c\epsilon} G(y, \epsilon) &= \int_0^1 dy y^{\beta+c\epsilon} \left[G(y, \epsilon) - \sum_{n=0}^N \frac{y^n}{n!} \partial_y^n G(0, \epsilon) \right] \\ &+ \sum_{n=0}^N \frac{\partial_y^n G(0, \epsilon)}{n! (\beta + n + 1 + c\epsilon)}. \end{aligned} \quad (86)$$

The first line is finite at $y = 0$, while the second line contains the explicit ϵ -poles.

For several endpoint axes, introduce the Taylor projector

$$(\mathcal{T}_a G)(y, \epsilon) = \sum_{n=0}^{N_{s,a}} \frac{y_a^n}{n!} [\partial_{y_a}^n G(y, \epsilon)]_{y_a=0}, \quad \mathcal{R}_a = 1 - \mathcal{T}_a. \quad (87)$$

Then the recursive subtraction identity can be written compactly as

$$\begin{aligned} I_s(\epsilon) &= \sum_{B \subseteq A_s} \sum_{\{0 \leq n_b \leq N_{s,b}\}_{b \in B}} \left[\prod_{b \in B} \frac{1}{n_b! (\beta_{s,b} + n_b + 1 + c_{s,b}\epsilon)} \right] \\ &\times \int dy_{\bar{B}} \left[\prod_{a \in A_s \setminus B} y_a^{\beta_{s,a} + c_{s,a}\epsilon} \right] \left[\prod_{a \in A_s \setminus B} \mathcal{R}_a \right] \left[\prod_{b \in B} \partial_{y_b}^{n_b} G_s(y, \epsilon) \right]_{y_B=0}. \end{aligned} \quad (88)$$

In the logarithmic case $\beta_{s,a} = -1$, one has $N_{s,a} = 0$, and the identity reduces to

$$I_s(\epsilon) = \sum_{B \subseteq A_s} \left[\prod_{b \in B} \frac{1}{c_{s,b}\epsilon} \right] \int dy_{\bar{B}} \left[\prod_{a \in A_s \setminus B} y_a^{-1+c_{s,a}\epsilon} \right] \\ \times \left[\prod_{a \in A_s \setminus B} (1 - \mathcal{T}_a) \right] [G_s(y, \epsilon)]_{y_B=0}, \quad (\mathcal{T}_a G)(y, \epsilon) = G(y, \epsilon)|_{y_a=0}. \quad (89)$$

Finally, expanding

$$G_s(y, \epsilon) = \sum_{k \geq 0} \epsilon^k G_s^{(k)}(y), \quad \prod_{a \in A_s \setminus B} y_a^{c_{s,a}\epsilon} = \sum_{r \geq 0} \frac{\epsilon^r}{r!} \left(\sum_{a \in A_s \setminus B} c_{s,a} \log y_a \right)^r, \quad (90)$$

gives the Laurent coefficients explicitly as ordinary finite integrals.

This audit leads to a useful interface boundary. A topology definition must retain the global parametric metadata: loop count, propagator powers, dimension, prefactor description, and affine powers of $\mathcal{U}$ and $\mathcal{F}$. A sector definition must retain the map, regular sector prefactor, monomial powers of $\mathcal{U}$ and $\mathcal{F}$, monomial powers from the Jacobian/measure or numerator, and the singular axes. The generic processor can then assemble $\rho_{s,a}$, evaluate ψ_s and ϕ_s from dualized black-box evaluators, and apply the appropriate localized subtraction formula.

The current numerical implementation still evaluates only the coefficients $\epsilon^{-2}, \epsilon^{-1}, \epsilon^0$ and implements explicit formulas for zero, one, and two logarithmic endpoint axes. Higher-rank endpoint structures and non-logarithmic endpoint powers require a recursive subtraction engine and a variable-length Laurent accumulator. The metadata described above is now present so such a recursive engine can be added without changing the black-box/topology/sector boundary.

5 Outlook: Schwinger sampling with momentum-space numerators

The construction above assumes that the loop momenta have already been integrated out, so all non-trivial information enters through the Symanzik polynomials and possible parameter-space numerator factors. A complementary path is useful when the denominator part should still be organized in Feynman/Schwinger parameters, but the numerator is easier to keep as a momentum-space black box. Schematically, start from a regulated integral

$$I[f] = \int d^{DL}k \frac{f(k)}{\prod_{e \in E} (D_e(k) + i0)^{\nu_e}}. \quad (91)$$

Using Schwinger parameters, or equivalently the projective Feynman-parameter form of the same denominator product,

$$\prod_{e \in E} \frac{1}{(D_e(k) + i0)^{\nu_e}} = \frac{\Gamma(\nu)}{\prod_e \Gamma(\nu_e)} \int_{\mathbb{P}_+^{|E|-1}} \prod_e dx_e x_e^{\nu_e-1} \left[\sum_e x_e (D_e(k) + i0) \right]^{-\nu}, \quad \nu = \sum_e \nu_e, \quad (92)$$

one obtains an integral over positive edge parameters and loop momenta, without replacing $f(k)$ by tensor-reduction formulae. The quadratic denominator combination can be written schematically as

$$\sum_e x_e D_e(k) = (k - Q(x))^T A(x) (k - Q(x)) + \mathcal{V}(x), \quad \mathcal{V}(x) = \frac{\mathcal{F}(x)}{\mathcal{U}(x)}, \quad (93)$$

with the appropriate $+i0$ prescription or later contour deformation understood. Introducing an overall Schwinger scale and auxiliary momentum variables then gives a representation of the schematic form

$$I[f] \propto \int_{\mathbb{P}_+^{|E|-1}} \prod_e dx_e x_e^{\nu_e-1} \frac{1}{\mathcal{U}(x)^{D/2} \mathcal{V}(x)^\omega} \times \int_0^\infty d\lambda \lambda^{\omega-1} e^{-\lambda} \int d\mu(q) f(k(x, q, \lambda)), \quad (94)$$

where

$$\omega = \sum_e \nu_e - \frac{DL}{2}. \quad (95)$$

Here $d\mu(q)$ denotes a fixed reference measure for the auxiliary momentum variables after the quadratic form has been normalized. The important feature is not the particular choice of that reference measure. The important feature is that the numerator remains a callable function of momenta, while the Feynman-parameter part still exposes $\mathcal{U}$, $\mathcal{F}$, endpoint monomials, and sector maps to the same FSD machinery as before.

After sector decomposition of the parameter variables, the intended structure is

$$I[f] = \sum_s \int_{[0,1]^{d_s}} dy \int d\lambda d\mu(q) \prod_{a \in A_s} y_a^{\rho_{s,a}(\epsilon)} G_s(y, \lambda, q; \epsilon) f(k_s(y, \lambda, q)). \quad (96)$$

The subtraction operators act on the sector variables y_a . The auxiliary variables λ and q are spectators for the endpoint algebra:

$$\mathcal{S}_s [G_s(y, \lambda, q; \epsilon) f(k_s(y, \lambda, q))], \quad (97)$$

with $\mathcal{S}_s$ the same local Taylor-subtraction operator used for the ordinary parameter-space integrand. Thus the future topology object would store not only $\mathcal{U}$, $\mathcal{F}$, and the parametric exponents, but also the momentum routing, the quadratic denominator data, and the map from sampled auxiliary variables to loop momenta. The integrand definition would expose a momentum-space numerator callback

$$f : k \mapsto f(k), \quad (98)$$

and FSD would provide

$$z \in [0, 1]^n \mapsto (K_{\text{FSD}}(z), k(z)), \quad (99)$$

so that the numerical integrand is $K_{\text{FSD}}(z)f(k(z))$.

This is conceptually different from momentum-space tropical sampling, such as the approach implemented in `momtrop`; see <https://github.com/alpha100p/momtrop>. The FSD version would deliberately use sector decomposition and subtraction for the Feynman-parameter part. It therefore inherits the larger sector count and will not compete with tropical sampling on the factorial-sector problem for finite parameter integrals. The advantage is different: the same framework can expose endpoint poles in ϵ , and, once contour deformation or another threshold treatment is added, it is not tied to a finite Euclidean integration domain. For generic higher-order endpoint subtractions one may also need derivatives of the composed numerator $f(k_s(y, \lambda, q))$, or an explicit regularity contract showing that only boundary values are required. This is a boundary condition for the future backend, not part of the current scalar implementation. Moreover, the momentum-space numerator must be understood as a D -dimensional numerator, not merely a four-dimensional one. Rational terms from the $(D-4)$ -dimensional components have to be retained, for example through the usual μ -term bookkeeping

$$k_{[D]}^2 = k_{[4]}^2 + \mu^2, \quad \mu^2 \equiv k_{[D-4]}^2, \quad (100)$$

with signs and metric conventions fixed consistently with the chosen denominator prescription.

6 Implementation in FSD

The repository contains the modular implementation used for the numerical checks described above. It is referred to simply as FSD. The executable entry point is

$$\text{FSD.py.} \quad (101)$$

A local virtual environment is created at

$$\text{.venv,} \quad (102)$$

and OneLOopBridge is required as an external dependency. The benchmark values are obtained from the Python binding calls

$$\text{oneloop_bridge.three_point}(0,0,s,m2,m2,m2), \quad (103)$$

$$\text{oneloop_bridge.four_point}(0,0,0,0,s12,s23,m2,m2,m2,m2). \quad (104)$$

The only implementation described here is the modular FSD code.

6.1 Code organization

The implementation is organized so that the black-box rule is visible directly in the code layout:

- **FSD.py** is the command-line steering layer. It parses options, validates kinematics, builds the request object, prints summaries, calls the benchmark, launches the integration, and renders the final table.
- **sectors_generator.py** owns the declarative sector data: sector maps X_s , regular sector prefactors, endpoint monomials $M_{U,s}$ and $M_{F,s}$, monomial powers from Jacobian/measure or numerator factors, singular axes, and generated Symbolica callbacks for these expressions.
- **integrand.py** owns the topology definition and the generic **SectorProcessor**. The topology retains $\mathcal{U}$ and $\mathcal{F}$ for display and evaluator construction, but the processor never substitutes a sector map into them symbolically.
- **integrator.py** owns the Havana sampling loop, manual Laurent coefficient accumulation, progress reporting, target-accuracy stopping, and hot-path timing profile.
- **benchmark.py**, **formatting.py**, **definitions.py**, and **utils.py** isolate OneLOopBridge access, colored output, shared data containers, and parenthesis uncertainty formatting.
- **dot_topology.py** is the scaffold for future GammaLoop-convention DOT-file input. It is wired into the same topology and sector interfaces, but its parser, topology builder, sector issuer, and benchmark mapping intentionally raise **NotImplementedError**.

6.2 DOT-file extension path

The command-line option

$$\text{--dot-file path/to/integral.dot} \quad (105)$$

selects the future GammaLoop DOT-backed topology path. The current code validates that the file exists and then dispatches to **GammaLoopDotTopologyBuilder**. The intended development sequence is

$$\text{DOT file} \longrightarrow \text{GammaLoopTopologyData}, \quad (106)$$

$$\text{GammaLoopTopologyData} \longrightarrow \text{TopologyDefinition}, \quad (107)$$

$$\text{GammaLoopTopologyData} \longrightarrow \{\text{SectorDefinition}\}. \quad (108)$$

This keeps arbitrary future topologies on the same path as the built-in triangle and box examples: topology-specific information is converted into retained Symanzik expressions and declarative sector metadata, while the `SectorProcessor` remains generic and continues to consume only black-box evaluators and sector callbacks.

6.3 Prepared sectors and black-box processing

The code mirrors the split used in this note. Sectors are prepared independently of the Symanzik polynomials. Each sector stores only

$$\{X_s, J_{s,\text{reg}}, M_{U,s}, M_{F,s}, \text{measure/numerator monomial powers, singular axes}\}, \quad (109)$$

plus Symbolica evaluators generated from these known sector expressions. The processing layer receives these prepared sectors together with black-box Symbolica evaluators

$$\hat{\mathcal{U}}(x; p), \quad \hat{\mathcal{F}}(x; p), \quad (110)$$

where p denotes the external invariants and masses. After the evaluators are built, the processing code does not inspect or symbolically rewrite the expressions for $\mathcal{U}$ or $\mathcal{F}$.

At runtime the processor receives only a `SectorDefinition`, a `TopologyDefinition`, and numeric sample points. For a sector point y it evaluates

$$x = X_s(y), \quad \hat{\mathcal{U}}(x), \quad \hat{\mathcal{F}}(x), \quad J_s(y), \quad (111)$$

through prebuilt Symbolica evaluators. If the sector has no endpoint monomial, the contribution is evaluated directly. If a monomial is declared, the residuals

$$\psi_s(y) = \frac{\hat{\mathcal{U}}(X_s(y))}{M_{U,s}(y)}, \quad \phi_s(y) = \frac{\hat{\mathcal{F}}(X_s(y))}{M_{F,s}(y)} \quad (112)$$

are evaluated by these quotients in the interior. On endpoint samples the same quantities are recovered from Taylor coefficients of dualized black-box $\hat{\mathcal{U}}$ and $\hat{\mathcal{F}}$ evaluators. The dual jets of the known sector map provide the chain-rule input, so no symbolic expansion of $\mathcal{U}(X_s(y))$ or $\mathcal{F}(X_s(y))$ is performed. The scalar $\hat{\mathcal{U}}$ and $\hat{\mathcal{F}}$ evaluators are constructed once with the topology. Dualized evaluators are cached by their requested Taylor shape and are obtained by copying these scalar evaluators before applying Symbolica's dualization step.

The generic processor then constructs

$$g_s(y; \epsilon) = J_{s,\text{reg}}(y) \psi_s(y)^{p_U(\epsilon)} \phi_s(y)^{p_F(\epsilon)}, \quad (113)$$

assembles the endpoint powers $\rho_{s,a}(\epsilon)$, and applies the declared localized subtraction formula. Zero, one, and two logarithmic singular axes are supported explicitly. Higher-rank or non-logarithmic endpoint structures raise an error instead of being guessed.

6.4 Havana sampling and stopping

FSD uses Symbolica's Havana integrator as a sampler rather than as a single black-box scalar integrator. One discrete dimension chooses the sector and each sector has a continuous adaptive grid of the same dimension:

$$\text{discrete sector id} \quad \otimes \quad y \in [0, 1]^d. \quad (114)$$

Lower-support subtraction pieces are localized into the same highest-dimensional sample space by dummy unit-density dependence on unused variables. This avoids separate grids for bulk, edge, and corner pieces.

Each sampled point contributes to all Laurent coefficients through the same manual accumulator,

$$A_k = \frac{1}{N} \sum_{n=1}^N w_n I_{k,s_n}(y_n), \quad k \in \{-2, -1, 0\}. \quad (115)$$

Havana is trained only on the finite-part magnitude used as an importance function. The adaptive grid is cloned at the beginning of an iteration; batch training samples are accumulated into the clone and merged back into the live grid only at the end of a completed iteration.

The target-accuracy stop is evaluated after each accumulated batch, not only after a full Havana iteration. Thus the final returned coefficients contain all completed batches up to the stopping point. In parallel mode the driver keeps only a worker-sized window of batches in flight, so early stopping does not submit the entire iteration unnecessarily. The option `--batch-size` therefore controls both vectorization size and mid-iteration stopping granularity.

6.5 Timing profile

The timing profile reported by the progress bar and final table is a work-time profile:

$$T_{\text{prof}} = T_{\text{Python}} + T_{\text{eval}} + T_{\text{Havana}}. \quad (116)$$

Here T_{eval} is the sum of Symbolica evaluator wall times over all workers, T_{Python} is NumPy and Python glue on workers plus master-side submission, result retrieval, accumulation, and progress rendering, and T_{Havana} is Havana sampling, training accumulation, grid merge, and grid update time. The displayed evaluator average is

$$\langle t_{\text{eval}} \rangle = \frac{10^6 T_{\text{eval}}}{N_{\text{accepted}}} \quad \text{microseconds per sample per worker}, \quad (117)$$

because T_{eval} is already summed over workers.

6.6 Implemented modes and conventions

For the triangle, the command-line interface supports massive and massless modes. In the validated finite massive region, endpoint monomial factorization is not applied. The implementation evaluates

$$C_0(s; m^2) = - \sum_{a=1}^2 \int_0^1 dt \int_0^1 dz J_{S_a}(t, z) \widehat{\mathcal{U}}(X_{S_a}(t, z))^{-1} \widehat{\mathcal{F}}(X_{S_a}(t, z))^{-1}. \quad (118)$$

In massless Euclidean mode, the implementation uses the subtraction formula of Section 3.2 and returns the coefficients of

$$C_0(s; 0) = \frac{C_{-2}}{\epsilon^2} + \frac{C_{-1}}{\epsilon} + C_0 + \mathcal{O}(\epsilon). \quad (119)$$

The default output convention is the OneLOopBridge convention. In this convention the universal Gamma/Euler factors are stripped from the displayed Laurent coefficients. For massless endpoint representations this also amounts to applying the finite convention shift described in the direct construction. The option `--prefactor-convention` controls only the overall scalar-integral normalization shown in the output table. The default value `raw` displays the raw OneLOopBridge normalization. The value `feynman` multiplies both the FSD coefficients and the benchmark coefficients by

$$\text{TO_FEYNMAN} = -\frac{1}{16\pi^2}. \quad (120)$$

In either display convention, values with Monte Carlo uncertainty are printed with the two-significant-digit parenthesis notation.

6.7 Implemented box example

The implementation also contains a scalar box example with massless external legs,

$$D_0(0, 0, 0, 0, s_{12}, s_{23}; m^2). \quad (121)$$

For equal internal masses, the simplex polynomial used by the implementation is

$$\mathcal{U} = x_0 + x_1 + x_2 + x_3, \quad \mathcal{F} = m^2 \mathcal{U}^2 - s_{12} x_0 x_2 - s_{23} x_1 x_3 - i0. \quad (122)$$

After imposing $\mathcal{U} = 1$, the finite four-point integral in $D = 4$ is

$$D_0 = \sum_{b=0}^3 \int_0^1 dy_0 \int_0^1 dy_1 \int_0^1 dy_2 J_b(y) \hat{\mathcal{F}}(X_b(y))^{-2}. \quad (123)$$

Here b labels the primary sector in which x_b is the largest Feynman parameter. In sector b ,

$$x_b = \frac{1}{1 + y_0 + y_1 + y_2}, \quad x_{j \neq b} = \frac{y_j}{1 + y_0 + y_1 + y_2}, \quad (124)$$

with the three y_j 's assigned in increasing order to the non-dominant parameters. The Jacobian is

$$J_b(y) = \frac{1}{(1 + y_0 + y_1 + y_2)^4}. \quad (125)$$

For $m = 0$, each primary sector has the form

$$\mathcal{F}(X_b(y)) = \frac{A_b y_\ell + B_b y_r y_s}{(1 + y_0 + y_1 + y_2)^2}, \quad (126)$$

where one of the two bilinears $x_0 x_2$ and $x_1 x_3$ contains the dominant parameter and is therefore linear in the primary variables. The implementation does not need the symbolic values of A_b and B_b ; it only uses the known positions of the linear variable y_ℓ and product variables y_r, y_s . The three secondary sectors are

$$L : \quad y_r = uv, \quad y_\ell = u, \quad y_s = w, \quad M_L = u, \quad (127)$$

$$P : \quad y_r = u, \quad y_s = v, \quad y_\ell = uvw, \quad M_P = uv, \quad (128)$$

$$Q : \quad y_r = u, \quad y_\ell = uv, \quad y_s = vw, \quad M_Q = uv. \quad (129)$$

The secondary Jacobian equals the displayed monomial in each case, so the regular prefactor after monomial extraction is again

$$\frac{1}{(1 + y_0(u, v, w) + y_1(u, v, w) + y_2(u, v, w))^4}. \quad (130)$$

The residual coefficient

$$\phi_{b\alpha}(u, v, w) = \frac{\mathcal{F}(X_{b\alpha}(u, v, w))}{M_\alpha(u, v, w)} \quad (131)$$

is evaluated by the black-box $\mathcal{F}$ away from the endpoint and by dual-number Taylor coefficients at $u = 0$ and $v = 0$. The one-monomial sector L uses the one-variable subtraction formula, while the P and Q sectors use the two-variable subtraction formula.

The raw coefficients produced directly by the massless box sectors are converted to the OneLOopBridge stripped convention as

$$D_{-2} = A_{-2}, \quad D_{-1} = A_{-1} + A_{-2}, \quad D_0 = A_0 + A_{-1} + \frac{\pi^2}{6} A_{-2}. \quad (132)$$

The current massless box command validates Euclidean kinematics, $s_{12} < 0$ and $s_{23} < 0$. Non-Euclidean massless kinematics are outside the present implementation.

6.8 Example commands

The massive triangle check can be run with

```
.venv/bin/python FSD.py --integral triangle --s 1.0 --m 1.0. (133)
```

The same check in the Feynman-normalized prefactor convention can be displayed with

```
.venv/bin/python FSD.py --integral triangle --s 1.0 --m 1.0  
--prefactor-convention feynman. (134)
```

The massless Euclidean triangle check can be run with

```
.venv/bin/python FSD.py --integral triangle --s -1.0 --m 0.0. (135)
```

The massive box example can be run with

```
.venv/bin/python FSD.py --integral box --s12 0.5 --s23 0.7 --m 1.0. (136)
```

The massless Euclidean box check can be run with

```
.venv/bin/python FSD.py --integral box --s12 -1.0 --s23 -2.0 --m 0.0. (137)
```

Massless timelike commands, such as `--integral triangle --s 1.0 --m 0.0`, are rejected by validation. OneLoopBridge is required in all runs because the implementation always compares the Laurent coefficients to a benchmark value. The implementation intentionally imports neither `scipy` nor `sympy`; all symbolic handling is done through Symbolica evaluators.